The overhead compounds quickly. A single user request may trigger five internal service calls. Each call involves DNS resolution, TCP connection establishment, HTTP request/response cycles, JSON serialisation, and network latency. What started as one external request becomes dozens of internal operations.

Understanding where overhead occurs is the first step towards elimination. Network latency, while unavoidable, can be minimised. Database queries can be optimised. Serialisation can be streamlined. Caching can eliminate redundant operations entirely.

Common sources of performance bottlenecks

Database query inefficiency ranks among the top performance killers in Django applications. The infamous N+1 query problem occurs when code fetches a list of objects and then makes separate database queries for each object's related data. For 100 products, this means 101 queries instead of 2. Each query adds round-trip latency to the database server, even if individual queries are fast.

Heavy payload sizes slow network transmission significantly. Sending complete user profiles with 50 fields when only names and email addresses are needed wastes bandwidth and processing time. JSON serialisation of large objects compounds this problem—converting Python objects to JSON strings requires CPU cycles proportional to data size.

Synchronous blocking operations tie up web server processes. When Django waits for an external API, generates a PDF report, or processes an uploaded image, that worker cannot handle other requests. Gunicorn or uWSGI servers have a limited number of workers. Under high load, all workers become occupied, and new requests queue up or time out.

Missing cache layers force repeated expensive operations. Calculating the same recommendation algorithm 10,000 times per day, executing identical database queries for every user, or fetching unchanged external data wastes resources that caching could preserve. Memory access is thousands of times faster than disk access.

Database connection overhead impacts high-concurrency scenarios. Establishing new database connections for every request consumes time and resources—PostgreSQL must authenticate, allocate memory, and initialise session variables. Without connection pooling, applications struggle to handle traffic spikes efficiently.

Uncompressed responses waste bandwidth, particularly on mobile networks. A 500KB JSON response might compress to 100KB with GZIP, reducing transmission time by 80%. For users on slow connections, this difference is noticeable.

Why Django excels for microservices

Django may seem heavyweight for microservices compared to Flask or FastAPI, but its mature ecosystem provides significant advantages for production systems.

The Django ORM abstracts database operations while offering sophisticated optimisation tools. Methods like select_related and prefetch_related eliminate N+1 queries with minimal code changes. Query analysis tools reveal performance issues during development. You can write clean Python code while maintaining control over generated SQL.

Django's middleware architecture efficiently handles cross-cutting concerns like authentication, logging, and request modification. Custom middleware can implement caching strategies, request tracking, and performance monitoring transparently across all views.

The framework's built-in caching system integrates seamlessly with Redis, Memcached, or database backends. From per-view caching to template fragment caching, Django provides flexible options for every use case without requiring external libraries.

Django Channels extends Django beyond HTTP to handle WebSockets, enabling real-time features like chat, notifications, and live dashboards. This makes Django suitable for both traditional APIs and modern real-time applications within a unified framework.

Django's integration with Celery, Redis, PostgreSQL, and NGINX—all open source, production-ready tools—creates a powerful stack for building scalable microservices on Linux systems. The Django community has solved common problems repeatedly, documenting solutions and creating reusable packages.

Security features like CSRF protection, SQL injection prevention, and XSS mitigation come built in. For microservices handling sensitive data, these protections are essential and well-tested in Django's codebase.

The admin interface, while often considered a monolith feature, remains valuable for internal tools and debugging in microservices. Operations teams can inspect data, trigger actions, and monitor system state without building custom interfaces.

Mastering database query optimisation

Database interactions often dominate request processing time. A poorly optimised query can take seconds instead of milliseconds, creating unacceptable user experiences. Django's ORM provides powerful tools to address this challenge, but developers must use them intentionally.

Fetching only required fields dramatically reduces data transfer and serialisation overhead. Instead of retrieving entire user objects with dozens of fields—many unused—specify exactly what you need.